Assignment 2

2026/02/14

Before you start

This assignment is almost entirely programming with a few written questions that you will answer based on the code you write. Hence, most of the instructions are in the starter code for this assignment.

In your GCP account, you can create a `n1-standard-8` with Operating System as Ubuntu 24.04 LTS (x86/64, amd64). We recommend you create the VM in `us-west-1a`, because the assignment of socket programming has a timeout limit, and you can easily fail if your client program suffers long latency from the server.

Here, we'll summarize information that is not in the provided code. The program will mainly use python3. But instead of directly installing python3 on your machine, we recommend using conda to create a clean and isolated environment. Conda is a very useful tool for python developers. Later if you want to use other versions of python, you do not need to interfere with the already-installed environment). You can learn conda basics at `https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html`

In addition to python3, you'll should install the python libraries for matplotlib, a graph plotting library. You should be able to install matplotlib by following the instructions here: `https://matplotlib.org/users/installing.html`. After you have created a conda environment, you can directly run `pip install -r requirements.txt` to install matplotlib.

When writing the code for the tasks, provide as much detail and comments as you think is appropriate, but err on the side of providing more rather than less detail for a question. In the case of a regrade request, we'll read and consider everything you have turned in.

The assignment will also teach you how to develop protocols using a network *simulator*, a computer program that imitates the behavior of the essential components of a network. For this assignment, we'll be using a custom simulator developed solely for assignment 2. I'll list some of the non-obvious details of the simulator here because they might help you when debugging your solutions to assignment 2.

Please **only edit** the code that is part of the TODO's. In particular, make use of all the attributes listed in the classes and do not remove them. Also avoid using custom imports that are not part of the standard Python distribution. You should not need them.

1 Socket Programming Practice: Rock, Paper, Scissors

1.1 Warm Up

You will play a game with the server. By running the client program

```
python3 tcp_client.py
```

You will connect to a server (we already launched the server). After the connection, the server will send a prompt, asking for your student ID. You must input your student ID correctly. After that, you will play "Rock,

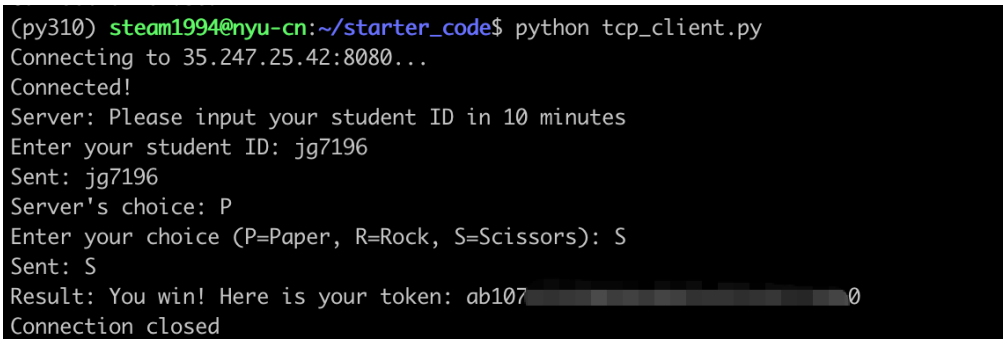

Figure 1: Example of Game Runtime

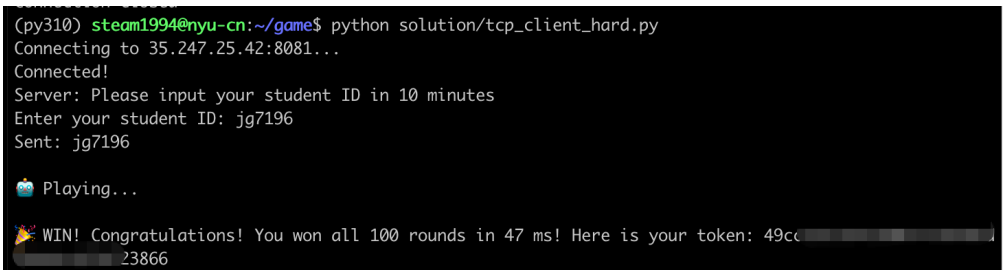

Figure 2: Example of Game Runtime (Hard)

Paper, Scissors" game with the server. The server will send you a letter $R|P|S$. You should also reply a letter to the server. If you win, the server will reply a token to you.

Then you create a txt file named game-1.txt, and it includes two lines: the first line is your student ID, the second line is the token provided by the server.

Figure 1 is how I played the game.

So I will create a file game-1.txt, and put two lines.

```
jg7196
ab107xxxxxxxxxxxxxxxxxxxxxxxxxxxx0
```

Each student will have a different student ID, so every one will have a different token.

1.2 Harder Game

In the second game, you need to read `tcp_client_hard.py` and complete the TODO. This time, you will play the game for 100 rounds, but the server has a strict timeout requirement. Every time the server sends you a P—R—S, you must reply to it within 10ms. That is why we recommend you put your VM in `us-west-1a` region, which is close to the server. Failing to do so, your program will easily trigger timeout and you cannot obtain the token finally.

Also, due to the timeout requirement, you can no longer manually type the answer for each round. Your program has to automatically respond to the server and defeat the server for 100 rounds. Also, to save time, you cannot print logs for every round. A successful client program (robot) will win the game for 100 rounds and obtain the final token.

Figure 2 is how I played the game.

So I will create a file game-2.txt, and put two lines.

```
jg7196
49cxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx3866
```

Each student will have a different student ID, so every one will have a different token.

You should compress game-1.txt and game-2.txt as a zip file and submit to gradescope. We have auto-grader to check whether your output is valid. In addition, your code (tcp_client_hard.py) will be submitted together withe the follow-up assignments to another gradescope assignment (We will manually check whether the code implementation is valid).

2 Moving Averages and Retransmission Timeouts

2.1 Overview of the Code

For this part, we'll start with the `TimeoutCalculator` found in `util/timeout_calculator.py`. This class will be used to determine how long each packet we create should wait before our host retrires sending it. To compute the timeout, we'll estimate the mean RTT and it's standard deviation using the EWMA-based algorithm we discussed in lecture 4.

We set `alpha`, `beta`, and `k` (which have the same function as α, β, and k from the lecture) during the construction of the calculator. Remeber, EWMA is a moving average, it changes over time as additional RTT data is added to the system. In our implementation, this is done in the `add_data_point()` function. When a host receives an acknowledgement, it can update our calculator with the latest RTT data point. This data point is then used to update our estimate of the mean RTT and RTT variance before updating the timestamp recommendation. One the `add_data_point()` function returns, the host can `timeout()` method to get the timeout recommendation based on the latest RTT data.

Notice that, when we update the timeout, we first make sure that the new timeout is within the timeout bounds we'll set during the beginning of the simluation. Why do we need a minimum value of the retransmission timeout? Why do we need a maximum value of the retransmission timeout? These questions might be easier to answer once you have incorporated the retransmission logic into both the Stop-And-Wait and sliding window protocols.

2.2 Tasks for the `TimeoutCalculator`

Your first task is to implement the helper functions used to compute the mean estimate, standard deviation estimate, and timeout.

- Open the `TimeoutCalculator` in `util/timeout_calculator.py`.
- First, implement the `__compute_new_mean_estimate()` function to estimate the mean using the previous estimate of the mean, the latest RTT datapoint, and α.
- Next, implement the `__compute_new_stddiv_estimate()` function to estimate the standard deviation using the previous estimate of the standard deviation, the mean, the latest RTT datapoint, and β.
- Finally, use your mean and standard deviation estimates, as well as k, to compute the new timeout recommendation in `__compute_timeout()`

2.3 Timeout Simulation

Once the `TimeoutCalculator` is completed, we can start running the timeout simulation. This simulation is run from `run_timeout_simulation.py`.

We have implemented a few different scenarios to test the `TimeoutCalculator` against. These scenarios produce a stream of mock RTT data that is added to your calculator over time. Each time a new data point is

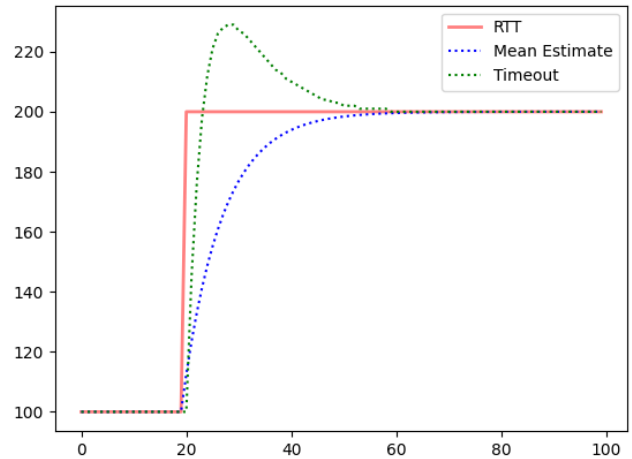

Figure 3: The result of a timeout simulation with $\alpha = 0.125$, $\beta = 0.25$, and $k = 1.5$ with a `permanent-change` scenario.

added, the simulator will fetch the updated mean estimate and timeout. Finally, it will plot these values so you can visualize the results.

Currently, we have four scenarios implemented in the `NetworkSimulator` class. Each scenario runs for 100 ticks and sends 1 new data point at each tick.

- `short-spike`: All packets have an RTT of 100, except for ticks 10 and 11, where it spikes up to 200.
- `long-spike`: Same as before, except the spike lasts from tick 10 to 29.
- `permanent-change`: The RTTs start our at 100, but at tick 20, they jump up to 200 where they remain.
- `high-variance`: The RTTs vary widly between 10 and 200.

The timeout simulation can be run using

```
python3 run_timeout_simulation.py  -a ALPHA  -b BETA  -k K  SCENARIO
```

To run the simulation, you must specify α, β, and the scenario. You can also specify k. For example, if we want to see how $\alpha = 0.125$, $\beta = 0.25$, and $k = 1.5$ fair against a permanent change in latency, we can run the simulation using

```
python3 run_timeout_simulation.py -a 0.125 -b 0.25 -k 1.5 permanent-change
```

If your timeout calculator was implemented correctly, you should see an image in `timeout_simulation.png` that looks like 3.

In your written report, please include the four different plots corresponding to each scenario. You can choose the same $\alpha = 0.125$, $\beta = 0.25$, and $k = 1.5$ for all scenarios.

3 Reliability

This next part of the assignment will have you implement a few of the reliability protocols we learned about in class. Specifically, we will implement the Stop-And-Wait and Sliding Window protocols to track outbound messages and retries, and we'll implement AIMD.

These will all run on the network simulator we mentioned at the beginning. We'll reveiw some of the details of that simulator before diving into the tasks.

3.1 Overview of the Simulator

Randomness in the simulator. The simulator uses a random number generator to generate independent and identically distributed (IID) packet losses[1] when packets are dequeued. This is in addition to packet drops if a queue overflows when packets are enqueued. You do not need to understand probability for this assignment, but you should be aware of the fact that randomness causes non-determinism in simulations. For instance, if you run two simulations with identical settings and the same loss rate, you might see different outputs because the sequence of packet drops will be randomly generated. To make such random simulations deterministic, you can use a *seed* to initialize a random number generator so that it generates the same sequence of random numbers. If you pass the same seed to the simulator in two different simulation runs with the same settings, the output will be the same. Also, if you don't use random losses in your simulation (i.e., you set the loss ratio to 0), then your output should be deterministic because no other simulator component uses a random number generator.

Sequence numbers. As a matter of convention, sequence numbers for packets start from 0 (i.e., the first packet has sequence number 0). Also, we'll be dealing with providing reliability at the level of packets, not bytes.

Link capacity in the simulator. The link capacity in all our simulations is fixed to 1 packet per simulation tick. This seems like an arbitrary restriction, but it simplifies the implementation of the simulation and the assignment, without losing anything in terms of what you will learn from the assignment. All time-based quantities (RTT, timeout, RTT_{min}, queuing delay, etc.) in this assignment are measured in units of simulation ticks, an arbitrary unit of time we use in our simulations.

RTT_{min} in the simulator. Because time is quantized to ticks in our simulation, an RTT_{min} of 10 ticks will result in the packet being in flight for 11 ticks. For example, if we send a packet from the host during tick 0, we'd expect to see that packet returned by `receive_all()` during tick 10.

Senders and receivers. Because this is a simulator, we get to simplify as much as we want, while still imitating network components relevant to what we're trying to study. Therefore, we do not actually implement a separate sender and receiver.

During this simulation, the host we implement will send a stream of data out to the network. In reality, this stream of data is just being looped back to the host (after being processed by the mock devices we'll talk about shortly). That said, when implementing the host, you should do so from the perspective of an engineer implementing a light-weight version of TCP streaming data to a remote server.

Treat all the data sent as messages, and all the packets received as acknowledgements from the remote server.

Devices. There are a few physical devices represented by different classes in the simulator. The most important of these for this assignment are the *hosts*, since those are your job To This is where we'll do the bulk of our work implementing the protocols.

The hardware device on most computers responsible for interacting with the network is the *network card*, or *network interface*. Each host has a network interface attached to it. The network interface contains egress (transmission) and ingress (reception) buffers. While the host is running, it should check the ingress buffers to see if any packets have been sent to this host. It should then figure out what data it wants to transmit, and add those packets to the egress buffer.

[1] In other words, every packet is dropped independent of every other packet, but we use the same probability to decide whether to drop a packet or not.

Next is the *clock*. To simplify things, our simulation has a global clock that tracks the current tick value. Any hardware device is able to read from this clock at any time to see what tick the simulation is currently on.

Then the *link*. When data is sent from the host over the network card, that packet's first stop will be the link. The link will add that packet to it's buffer. The link will then transmit the packet on the front of the buffer queue when there's available capacity (i.e., once per tick).

Finally, we have the *delay box*, which simulates the packet as it travels through the wide-area network (WAN). Specifically, this device holds the packet and releases it only after the specified RTT has passed. While the delay box is processing the packet, it also marks it as an ACK packet. This is also where packets are randomly dropped to represent packet loss. Luckily, since this is a simulation, we know with certainty when a packet was dropped and log its sequence number, making debugging easier.

Ticks. It might be helpful to review what exactly happens during each tick of the simulation. During each tick, the simulator first updates the clock. Next, it hands control of the simulation over to the host. During this time, the host should read all packets that are currently on the ingress buffer of the network interface and process the newly received messages. The host will then place any new outbound packets on the egress buffer of the network interface. Once the host finishes processing the messages, the simulator flushes the network interface's egress buffer to the link. It then flushes the link's buffer out to the WAN (the delay box). Finally, it any packets arriving from the WAN are placed onto the network interface's ingress buffer to be processed during the next tick.

3.2 Overview of the Code

Entry Point: `run_reliability_simulation.py`

This file is the script that starts the simulation. It just acts as an entry point. It parses command line arguments to determine how to configure the network. You must specify the RTT_{min} and the duration of the simulation (in number of ticks). Additionally, you can configure the simulation's loss rate, the maximum number of packets in the link's queue, and the timeout bounds. Finally, you can manually specify the seed for the random number generator, if you want to work with deterministic output.

Once you've passed in the arguments to configure the simulation, you can configure the host. Start by specifying which protocol the host will implement. This can be `stop-and-wait`, `sliding-window`, or `aimd`. Finally, on a host that implements the sliding window protocol, you must specify the window size. Example commands can be found in §6.

The script then build the network devices, before handing control over to the simulator itself. It prints the results of the simulation to the terminal. The simulator reports the max sequence number that has been received *in order* at the end of the simulation period. Adding 1 to this gives you the number of packets that have been received in order. Dividing the number of packets by the simulation period will give you the transport-layer throughput.

Network Card: `network/network_interface.py`

Since this simulation isn't interested in the actual data being transmitted (just the packets themselves), the packets only contain metadata. Specifically, each packet contains the sequence number of that message and the send timestamp.

Remeber, all network interactions from the host should happen through the network interface. To send a `packet` from the host, use the `network_interface.transmit(packet)` method.

Similarly, you can use `network_interface.receive_all()` to get a list of all packets that were sent to this host.

Hosts: `host/host.py`

This file contains the interface for the hosts. The interface represents the contract between the hosts and the simulator. Specifically, each host must implement the `run_one_tick()` method, which reads data off the network card, writes new data to the network card, and returns the latest in-order sequence number that's been acknowledged.

All the different host types that implement this interface are also defined in the `host/` directory.

3.3 Tasks for the Stop-and-Wait Protocol

- Fill in the TODOs in `host/stop_and_wait_host.py`. Specifically, complete the `run_one_tick()` function. You'll notice this function is broken into three stages
  1. Read all packets received on the network card. Process the acknowledgements.
  2. Determine which packets haven't been acknowledged, but have timeout. Then, retransmit those messages.
  3. Determine if any new packets can be trasmitted. If they can be, transmit them.
- Complete the conceptual questions in your written report. Note: All hosts will implement the same three stages during their execution of each tick.

Testing the Stop-and-Wait Protocol Implementation

Use the simulator's default large queue size limits (1M packets) for this experiment.

1. Make sure your implementation works. This means checking that a packet is being sent out every RTT_{min} ticks. Print out a line to the terminal every time you send out a packet. This should help you debugging your submission.
2. Run the Stop-And-Wait protocol for several different values of RTT_{min}. Plot the throughput you get as a function of $\frac{1}{RTT_{min}}$. You can use matplotlib for this. As a *rough* estimation, the throughput can be predicted as $\frac{W}{RTT}$, W indicates how many packets are transmitted in 1 RTT. In Stop-And-Wait protocol, in every RTT, there is only 1 packet that can complete transmission.
3. Introduce a small amount of IID loss (about 1%).

   In your written report, check (1) if the throughput continues to be close to the value predicted by the equation and (2) that the protocol continues to function correctly despite the loss of packets. If the throughput is much less than the value predicted by $\frac{1}{RTT_{min}}$, explain why, by looking at when packets are originally transmitted and when they are retransmitted. During the presence of a small amount of IID loss (about 1%), does the divergence between the simulation's throughput and the equation's predicted throughput ($\frac{1}{RTT_{min}}$) increase or decrease as RTT_{min} increases? Why?

3.4 Tasks for the Sliding Window Protocol

- Finish the TODOs in `host/sliding_window_host.py`.
- Complete the conceptual questions in your written report.

Testing the Sliding Window Protocol Implementation

Use the simulator's default large queue size limits for this experiment. **In your written report,** answer the same three questions as the Stop-And-Wait protocol in the previous section, but remember to vary the window size as well in addition to RTT_{min}. When a small amount of loss (1%) is introduced, how does the divergence

between the simulation's throughput and the equation's predicted throughput vary now as a function of *both* RTT_{min} and the window size.

4 AIMD

In the final part of this assignment, you will implement the AIMD algorithm. As previously mentioned, this algorithm uses the sliding window protocol we've already implemented. The only major new parts of AIMD are implementing the Additive Increase and Multiplicative Decrease rules.

4.1 Tasks for the AIMD Protocol

- Finish the TODOs in `run_one_tick()` functions of `host/aimd_host.py`
- Complete the conceptual questions in your written report.

After you have properly completed the code, run

```
python run_reliability_simulation.py --seed 1 --rtt-min 10 --ticks 10000 --queue-limit 10 aimd
```

you should be able to generate a similar figure to Figure 4.

Testing the AIMD Implementation

In your written report, answer the following questions. If you need a concrete value of RTT_{min}, you can set it to 10 ticks for this experiment, but feel free to use your own value of RTT_{min} if you wish.

Now vary the queue-limit, and try different values from 1, 10, 20, 50. Include the 4 plots with your submission.

1. Do you see the additive increase, multiplicative decrease, sawtooth pattern in each plot? What is the period of the sawtooth? You can measure the period from the window size plot that demonstrates the sawtooth pattern. (You do not need to report a very accurate number, just give a rough estimation based on your plots)
2. What is the throughput of AIMD in this case? Our code will output a log in the end, telling you "Maximum in order received sequence number" that have been completed during the specified ticks.
3. AIMD needs a certain amount of queue capacity so that it achieves throughput close to the link's capacity. What is the purpose of this queue?

5 Final Check

Your submission include **all the coding files, as well as a PDF named report.pdf**. Make a zip file and submit to Gradescope: Assignment-2-Code-and-Report.

Make sure your have also submitted the game results (Section 1) to Gradescope: Assignment-2-Game (we have auto-grader configured there and you can immediately know whether your token is correct). When grading socket programming practice, we will check both the results from the autograder, and your code.

6 Sample output

We have provided some sample output below for you to confirm if your implementations of the Stop-And-Wait, Sliding Window, and AIMD protocols are working correctly. Note that your output may not match up exactly with these outputs because there is some flexibility in how you implement each protocol and there are no simple "unit tests" for congestion-control protocols. If you're unsure if your protocol is working correctly, ask the course staff.

6.1 Stop-And-Wait

```
python3 run_reliability_simulation.py --seed 1 --rtt-min 10 --ticks 50 stop-and-wait

host_type: stop-and-wait
rtt_min: 10
ticks: 50
seed: 1
loss_ratio: 0.0
queue_limit: 1000000
min_timeout: 100
max_timeout: 10000

-----------------------------------------
| Tick | Event Type | Event Description  |
|------|------------|--------------------|
|    0 |   Transmit | Sequence number: 0 |
|   10 |    Receive | Sequence number: 0 |
|   10 |   Transmit | Sequence number: 1 |
|   20 |    Receive | Sequence number: 1 |
|   20 |   Transmit | Sequence number: 2 |
|   30 |    Receive | Sequence number: 2 |
|   30 |   Transmit | Sequence number: 3 |
|   40 |    Receive | Sequence number: 3 |
|   40 |   Transmit | Sequence number: 4 |
-----------------------------------------

Maximum in order received sequence number 3
```

6.2 Sliding Window

```
python3 run_reliability_simulation.py --seed 1 --rtt-min 2 --ticks 10 sliding-window
--window-size 5

host_type: sliding-window
rtt_min: 2
ticks: 10
seed: 1
loss_ratio: 0.0
queue_limit: 1000000
min_timeout: 100
max_timeout: 10000
window_size: 5

--------------------------------------------
| Tick | Event Type | Event Description   |
|------|------------|---------------------|
|    0 |   Transmit | Sequence number: 0  |
|    0 |   Transmit | Sequence number: 1  |
|    0 |   Transmit | Sequence number: 2  |
|    0 |   Transmit | Sequence number: 3  |
|    0 |   Transmit | Sequence number: 4  |
|    2 |    Receive | Sequence number: 0  |
|    2 |   Transmit | Sequence number: 5  |
|    3 |    Receive | Sequence number: 1  |
|    3 |   Transmit | Sequence number: 6  |
|    4 |    Receive | Sequence number: 2  |
|    4 |   Transmit | Sequence number: 7  |
|    5 |    Receive | Sequence number: 3  |
|    5 |   Transmit | Sequence number: 8  |
|    6 |    Receive | Sequence number: 4  |
|    6 |   Transmit | Sequence number: 9  |
|    7 |    Receive | Sequence number: 5  |
|    7 |   Transmit | Sequence number: 10 |
|    8 |    Receive | Sequence number: 6  |
|    8 |   Transmit | Sequence number: 11 |
|    9 |    Receive | Sequence number: 7  |
|    9 |   Transmit | Sequence number: 12 |
--------------------------------------------

Maximum in order received sequence number 7
```

6.3 AIMD

See Figure 4. The figure was generated using the command line:

```
python3 run_reliability_simulation.py --loss-ratio 0.0 --seed 1 --rtt-min 20 --ticks 10000
--queue-limit 10 aimd

host_type: aimd
rtt_min: 20
ticks: 10000
seed: 1
loss_ratio: 0.0
queue_limit: 10
min_timeout: 100
max_timeout: 10000

-------------------------------------------------------------------------------------------
| Tick | Event Type                | Event Description                                    |
|------|---------------------------|------------------------------------------------------|
|    0 |                  Transmit | Sequence number: 0                                   |
|   20 |                   Receive | Sequence number: 0                                   |
|   20 |          Expanding Window | Old: 1.0, New: 2.0                                   |
|   20 |                  Transmit | Sequence number: 1                                   |
|   20 |                  Transmit | Sequence number: 2                                   |
|   40 |                   Receive | Sequence number: 1                                   |
|   40 |          Expanding Window | Old: 2.0, New: 3.0                                   |
|   40 |                  Transmit | Sequence number: 3                                   |
|   40 |                  Transmit | Sequence number: 4                                   |
|   41 |                   Receive | Sequence number: 2                                   |
|   41 |          Expanding Window | Old: 3.0, New: 4.0                                   |
|   41 |                  Transmit | Sequence number: 5                                   |
|   41 |                  Transmit | Sequence number: 6                                   |
|   60 |                   Receive | Sequence number: 3                                   |
|   60 |          Expanding Window | Old: 4.0, New: 5.0                                   |
|   60 |                  Transmit | Sequence number: 7                                   |
|   60 |                  Transmit | Sequence number: 8                                   |
|   61 |                   Receive | Sequence number: 4                                   |
...
| 9997 |                  Transmit | Sequence number: 7516                                |
| 9998 |                   Receive | Sequence number: 7505                                |
| 9998 |          Expanding Window | Old: 12.611559591457588, New: 12.690851925823383 |
| 9998 |                  Transmit | Sequence number: 7517                                |
| 9999 |                   Receive | Sequence number: 7506                                |
| 9999 |          Expanding Window | Old: 12.690851925823383, New: 12.76964884236176  |
| 9999 |                  Transmit | Sequence number: 7518                                |
-------------------------------------------------------------------------------------------

Maximum in order received sequence number 7506
```

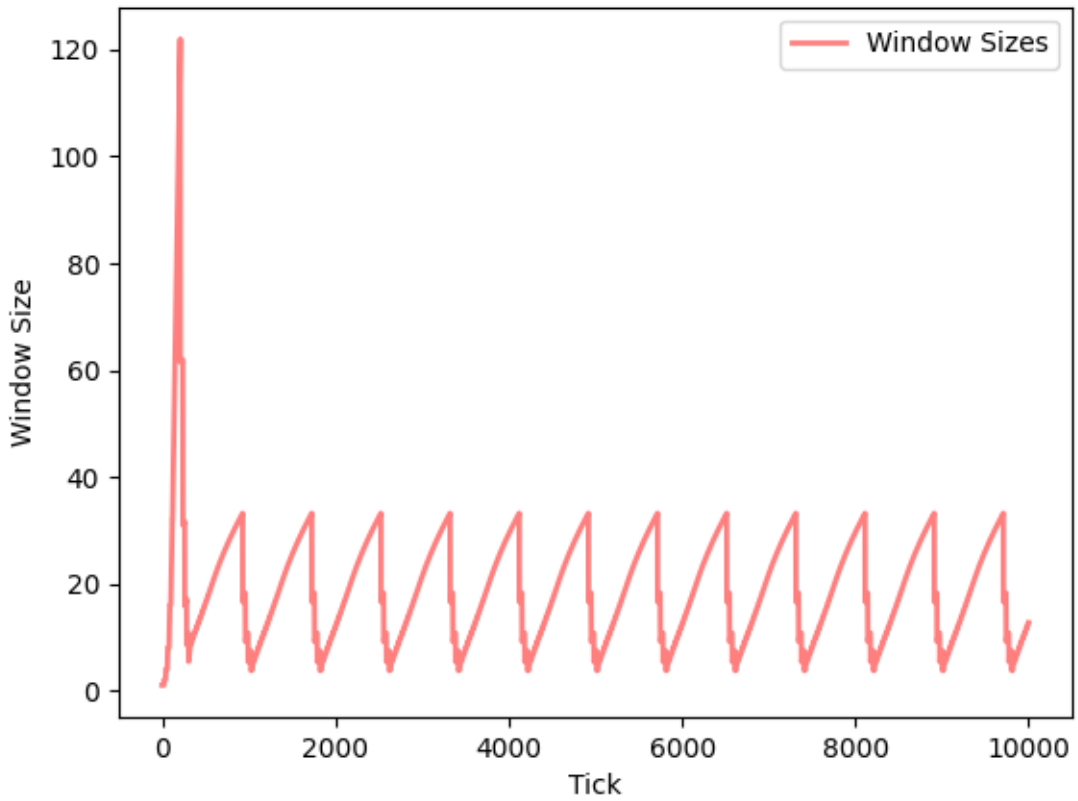

Figure 4: Evolution of window size when running the AIMD protocol. AIMD should set the window size W according to this pattern over time; the window size slowly ramps up (slow start) until reaching a threshold where it is larger than the network can allow, so AIMD quickly backs off to a much smaller baseline W.